

PHASEMATRIX-HIVEMIND-03.1

Dual-Fabric Field-Tensor Stitch Layer

Eigenstaendige formale Patch-Spezifikation fuer PhaseMatrix v0.3

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert einen additiven Patch fuer ein bereits vorhandenes PhaseMatrix-Hivemind-03-System. Der Patch fuehrt eine Dual-Fabric-Schicht ein, die eine freie, ephemere Resonanzlage (Fabric-H) mit einem persistenten Feldtensor (Fabric-T) verbindet. Der zentrale Mechanismus ist ein Stitcher: ResonanceCluster-Ereignisse duerfen den persistenten FieldTensorState nur dann veraendern, wenn ein fail-closed StitcherGate bestanden ist. Jede akzeptierte Kopplungsaenderung wird als content-addressed CouplingUpdate mit Trace, Evidence, Boundary-Report, Mirror-Consistency und Replay-Metadaten persistiert. Der Patch erzeugt keine Commits, keine finalen Artefakte und keine externen Effekte; er materialisiert ausschliesslich gegatete Strukturupdates im lokalen Matrixzustand.

Inhaltsverzeichnis

1	Status, Zweck und Architekturgrenze	3
1.1	Status	3
1.2	Zweck	3
1.3	Architekturgrenze	3
1.4	Nicht-Zweck	3
2	Systemposition	4
2.1	Schichtmodell	4
2.2	Integrationspunkt	4
3	Normative Invarianten	4
4	Kanonische Primitive	5
4.1	Run Descriptor Extension	5
5	Fabric-H: ResonanceFabricState	5
5.1	Semantik	5
5.2	Fabric-H-Invariante	6
6	Fabric-T: FieldTensorState	6
6.1	Semantik	6
7	StitchCandidate und CouplingUpdate	7
7.1	StitchCandidate	7
7.2	CouplingUpdate	7
8	MirrorConsistency und TensorDelta	8
8.1	MirrorConsistencyReport	8
8.2	TensorDeltaReport	8
9	StitcherGate	8
9.1	Gate-Formel	8
9.2	GateReport	8
10	StitcherReport und FieldTensorTrace	9
11	Pipeline	10
11.1	Referenzablauf	10
11.2	Pseudocode	10

12 CLI	10
13 Tests	11
13.1 Unit Tests	11
13.2 Integration Tests	11
13.3 Negative Tests	11
14 Definition of Done	11
15 Coding-Agent-Auftrag	12
16 Schlussinvariante	12

1 Status, Zweck und Architekturgrenze

1.1 Status

Dieses Dokument ist eine normative Patch-Spezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich fuer die Implementierung.

1.2 Zweck

Der Patch ergaenzt die vorhandene morphodynamische Zell- und Cluster-Schicht um eine klare Kopplungsmechanik zwischen:

- **Fabric-H**: ephemere, frei bewegliche Resonanzlage aus ResonanceClusters, ResonancePulses und transienten Links;
- **Fabric-T**: persistenter Feldtensor aus Node-Zustaenden, Kopplungsmatrix, Seam-Status und gespeicherten Tensor-Revisionen;
- **Stitcher**: fail-closed Materialisierungsoperator, der nur kohärente und begrenzte Kopplungsänderungen in Fabric-T schreibt.

Kernregel

Resonanz darf frei schwingen. Struktur darf nur gegatet nachziehen. Ein ephemeres Resonanzereignis darf den persistenten Feldtensor niemals direkt veraendern.

1.3 Architekturgrenze

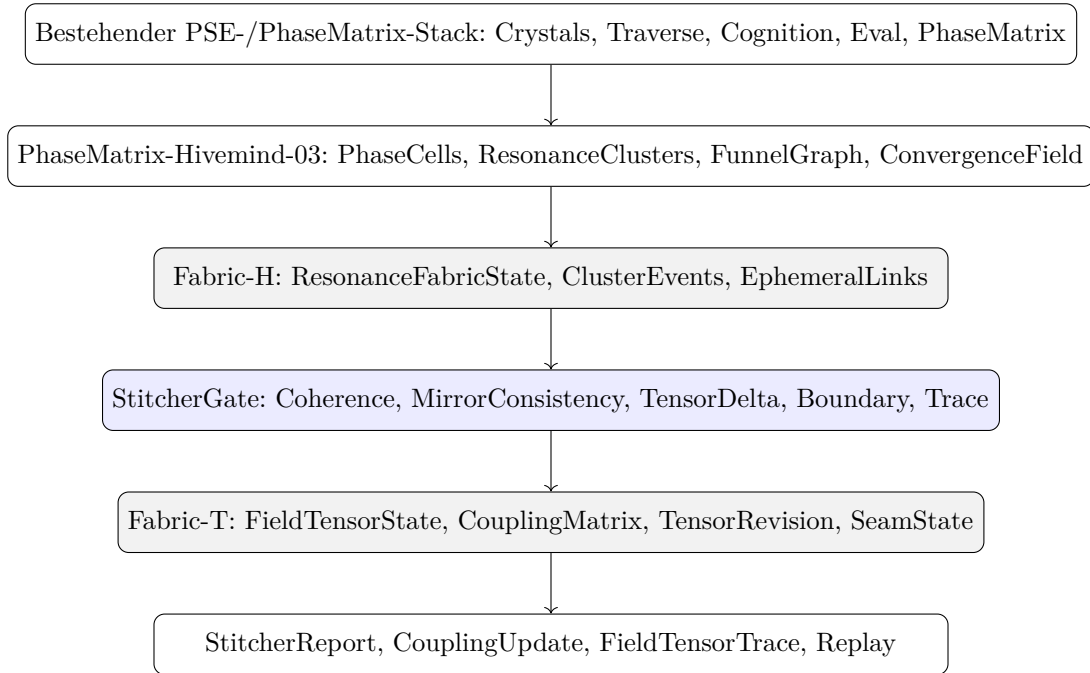
Der Patch ist kein Ersatz fuer die vorhandene PhaseMatrix-v0.3-Pipeline. Er setzt deren Kernobjekte voraus: PhaseCell, ResonanceCluster, FunnelGraph, ConvergenceField, IntentCandidate, ClusterTrace, Dissolution und MatrixBoundaryReport. Er fuegt keine neue Hivemind-Philosophie hinzu, sondern eine implementierbare Strukturupdate-Schicht.

1.4 Nicht-Zweck

Die Spezifikation definiert keinen Netzwerktransport, keine Verschluesselung, keine Privacy-Claims, keine externe Aktorik, keinen Commit-Pfad und keine finale Emission. Der Stitcher erzeugt ausschliesslich interne, auditierbare Tensor-Updates.

2 Systemposition

2.1 Schichtmodell



2.2 Integrationspunkt

Der Patch wird unter der vorhandenen Zellschicht implementiert:

```

crates/phase-matrix/src/cell/
  field_tensor.rs
  resonance_fabric.rs
  coupling_update.rs
  stitcher_gate.rs
  stitcher.rs
  tensor_delta.rs
  mirror_consistency.rs
  field_tensor_trace.rs
  
```

Die bestehende v0.3-Pipeline `cluster-cycle` bleibt gueltig. Der neue Stitcher kann nach `ClusterTrace` oder nach `Convergence/Intent` ausgefuehrt werden, sofern die Implementierung die erforderlichen Inputs bereitstellt.

3 Normative Invarianten

1. Fabric-H-Ereignisse **MUST NOT** Fabric-T direkt mutieren.
2. Jede Fabric-T-Aenderung **MUST** durch ein akzeptiertes `CouplingUpdate` erfolgen.
3. Jedes `CouplingUpdate` **MUST** content-addressed sein.
4. Jedes `CouplingUpdate` **MUST** auf genau einen `StitcherReport` verweisen.
5. Ein fehlender Trace **MUST** den Stitcher fail-closed machen.
6. Ein fehlender Boundary-Report **MUST** den Stitcher fail-closed machen.
7. Ein zu grosser Tensor-Delta **MUST** das Update ablehnen.

8. Abgelehnte Updates **MUST NOT** den FieldTensorState veraendern.
9. Replay **MUST** byte-identische StitcherReports reproduzieren.
10. Alle gate-relevanten Zahlen **MUST** fixed-point oder rational kanonisiert sein.

4 Kanonische Primitive

```
pub type Hash256 = [u8; 32];
pub type Fixed = CanonicalNumber;

pub struct EvidenceRef {
  pub kind: String,
  pub address: Hash256,
  pub relation: String,
}
```

4.1 Run Descriptor Extension

```
pub struct StitchRunDescriptor {
  pub run_id: String,
  pub parent_phase_matrix_run: Hash256,
  pub cell_substrate_report_hash: Option<Hash256>,
  pub operator_versions: BTreeMap<String, String>,
  pub thresholds: StitchThresholds,
  pub policies: StitchPolicies,
  pub canonicalization_version: String,
}

pub struct StitchThresholds {
  pub min_convergence_score: Fixed,
  pub min_mirror_consistency_index: Fixed,
  pub max_tensor_delta_norm: Fixed,
  pub max_coupling_delta_per_edge: Fixed,
  pub max_updates_per_cycle: u64,
  pub min_trace_completeness: Fixed,
}

pub struct StitchPolicies {
  pub require_boundary_pass: bool,
  pub require_trace_ready: bool,
  pub require_evidence_refs: bool,
  pub reject_empty_update_set: bool,
  pub sort_updates_before_hash: bool,
}
```

5 Fabric-H: ResonanceFabricState

5.1 Semantik

Fabric-H ist die ephemere Resonanzlage. Sie haelt nicht den persistenten Systemzustand, sondern die aktuelle, frei bewegliche Konstellation aus Clustern, Pulsen, transienten Links und Konvergenzfeldern.

```
pub struct ResonanceFabricState {
```

```

    pub fabric_h_id: Hash256,
    pub source_cycle_report_hash: Hash256,
    pub active_cluster_hashes: Vec<Hash256>,
    pub resonance_pulse_hashes: Vec<Hash256>,
    pub transient_links: Vec<EphemeralResonanceLink>,
    pub convergence_field_hashes: Vec<Hash256>,
    pub trace_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct EphemeralResonanceLink {
    pub link_id: Hash256,
    pub source_cell: Hash256,
    pub target_cell: Hash256,
    pub resonance_score: Fixed,
    pub phase_alignment: Fixed,
    pub ttl_ticks: u64,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

5.2 Fabric-H-Invariante

Fabric-H **MAY** Links enthalten, die nie persistiert werden. Fabric-H **MUST** jedoch immer einen Trace besitzen, der erklärt, aus welchen ClusterReports, ResonancePulses und ConvergenceFields die Link-Kandidaten stammen.

6 Fabric-T: FieldTensorState

6.1 Semantik

Fabric-T ist die persistente Kopplungsstruktur des Substrats. Sie repräsentiert, welche Zellen, Cluster oder Nodes strukturell gekoppelt sind und welche CouplingMatrix aktuell gilt.

```

pub struct FieldTensorState {
    pub tensor_id: Hash256,
    pub tensor_revision: u64,
    pub matrix_id: Hash256,
    pub node_state_hashes: Vec<Hash256>,
    pub spatial_topology_hash: Hash256,
    pub coupling_matrix_hash: Hash256,
    pub seam_state_hashes: Vec<Hash256>,
    pub previous_tensor_hash: Option<Hash256>,
    pub trace_head: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct CouplingMatrix {
    pub matrix_id: Hash256,
    pub entries: Vec<CouplingEntry>,
}

pub struct CouplingEntry {
    pub source: Hash256,
    pub target: Hash256,
    pub weight: Fixed,
}

```

```

    pub coupling_kind: CouplingKind,
    pub last_update_hash: Option<Hash256>,
}

pub enum CouplingKind {
    Structural,
    Resonance,
    Temporal,
    Semantic,
    Boundary,
}

```

7 StitchCandidate und CouplingUpdate

7.1 StitchCandidate

Ein StitchCandidate ist noch keine Tensor-Aenderung. Er ist eine vorgeschlagene Kopplungs-aenderung, die aus Fabric-H abgeleitet wurde.

```

pub struct StitchCandidate {
    pub candidate_id: Hash256,
    pub source_link_id: Hash256,
    pub source_cell: Hash256,
    pub target_cell: Hash256,
    pub proposed_coupling_kind: CouplingKind,
    pub proposed_delta: Fixed,
    pub reason_hash: Hash256,
    pub supporting_cluster_trace_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

7.2 CouplingUpdate

Ein CouplingUpdate ist die akzeptierte, persistente Aenderung an Fabric-T.

```

pub struct CouplingUpdate {
    pub update_id: Hash256,
    pub candidate_id: Hash256,
    pub source_cell: Hash256,
    pub target_cell: Hash256,
    pub old_coupling_hash: Option<Hash256>,
    pub new_coupling_hash: Hash256,
    pub delta_resonance: Fixed,
    pub delta_norm: Fixed,
    pub reason_hash: Hash256,
    pub gate_report_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

```


8 MirrorConsistency und TensorDelta

8.1 MirrorConsistencyReport

MirrorConsistency prueft, ob eine vorgeschlagene Kopplungsänderung mit der Spiegel-/Gegenprojektion kompatibel ist. Diese Schicht verhindert, dass ein lokales Resonanzereignis eine einseitige Strukturmutation ausloest.

```
pub struct MirrorConsistencyReport {
    pub report_id: Hash256,
    pub candidate_id: Hash256,
    pub source_projection_hash: Hash256,
    pub mirror_projection_hash: Hash256,
    pub mirror_consistency_index: Fixed,
    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

8.2 TensorDeltaReport

TensorDelta begrenzt die Groesse der Strukturveraenderung pro Zyklus.

```
pub struct TensorDeltaReport {
    pub report_id: Hash256,
    pub tensor_before_hash: Hash256,
    pub proposed_tensor_after_hash: Hash256,
    pub delta_norm: Fixed,
    pub max_allowed_delta_norm: Fixed,
    pub per_edge_delta_max: Fixed,
    pub update_count: u64,
    pub passed: bool,
}
```

9 StitcherGate

9.1 Gate-Formel

$$G_{stitch} = G_{conv} \wedge G_{mci} \wedge G_{delta} \wedge G_{budget} \wedge G_{trace} \wedge G_{boundary} \wedge G_{evidence}.$$

Mit:

$$\begin{aligned} G_{conv} &:= convergence_score \geq \theta_{conv}, \\ G_{mci} &:= mirror_consistency_index \geq \theta_{mci}, \\ G_{delta} &:= tensor_delta_norm \leq \theta_{delta}, \\ G_{budget} &:= update_count \leq max_updates_per_cycle. \end{aligned}$$

9.2 GateReport

```
pub struct StitcherGateReport {
    pub report_id: Hash256,
    pub candidate_id: Hash256,
    pub convergence_score: Fixed,
```

```

    pub mirror_consistency_index: Fixed,
    pub tensor_delta_norm: Fixed,
    pub update_budget_ok: bool,
    pub trace_ready: bool,
    pub boundary_passed: bool,
    pub evidence_ready: bool,
    pub passed: bool,
    pub failure_policy: StitchFailurePolicy,
}

pub enum StitchFailurePolicy {
    Hold,
    RejectCandidate,
    KeepTensorUnchanged,
    RequireRecompute,
    BoundaryViolation,
    DeterminismViolation,
}

```

Fail-Closed-Regel

Wenn irgendein Teilgate fehlschlaegt, **MUST** der FieldTensorState byte-identisch unveraendert bleiben. Der Fehler **MUST** als SticherGateReport und SticherReport persistiert werden.

10 SticherReport und FieldTensorTrace

```

pub struct SticherReport {
    pub report_id: Hash256,
    pub rd_hash: Hash256,
    pub fabric_h_hash: Hash256,
    pub tensor_before_hash: Hash256,
    pub tensor_after_hash: Hash256,
    pub stitch_candidates_hash: Hash256,
    pub accepted_updates: Vec<Hash256>,
    pub rejected_candidates: Vec<Hash256>,
    pub gate_reports: Vec<Hash256>,
    pub tensor_delta_report_hash: Hash256,
    pub field_tensor_trace_hash: Hash256,
    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct FieldTensorTrace {
    pub trace_id: Hash256,
    pub tensor_before_hash: Hash256,
    pub tensor_after_hash: Hash256,
    pub coupling_update_hashes: Vec<Hash256>,
    pub rejected_candidate_hashes: Vec<Hash256>,
    pub source_cluster_trace_hashes: Vec<Hash256>,
    pub boundary_report_hashes: Vec<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

11 Pipeline

11.1 Referenzablauf

1. Lade `StitchRunDescriptor`.
2. Lade aktuellen `FieldTensorState`.
3. Lade den letzten `CellSubstrateCycleReport` oder `ClusterTrace`-Satz.
4. Konstruiere `ResonanceFabricState` aus Clustern, Pulsen und transienten Links.
5. Leite `StitchCandidate[]` aus Fabric-H ab.
6. Berechne pro Kandidat `MirrorConsistencyReport`.
7. Erzeuge hypothetischen Tensor nach Kandidatenanwendung.
8. Berechne `TensorDeltaReport`.
9. Evaluiere `StitcherGateReport[]`.
10. Wende nur akzeptierte `CouplingUpdate[]` auf Fabric-T an.
11. Schreibe `FieldTensorTrace`.
12. Schreibe `StitcherReport`.
13. Replay validiert byte-identische Reports und Tensorrevisionen.

11.2 Pseudocode

```
pub fn run_stitch_cycle(
  rd: StitchRunDescriptor,
  tensor: FieldTensorState,
  cell_report: CellSubstrateCycleReport,
) -> StitcherOutcome {
  let fabric_h = build_resonance_fabric(&rd, &cell_report);
  let candidates = derive_stitch_candidates(&rd, &fabric_h, &tensor);
  let mirror_reports = evaluate_mirror_consistency(&rd, &candidates, &tensor);
  let proposed = simulate_tensor_updates(&rd, &tensor, &candidates, &mirror_reports);
  let delta = compute_tensor_delta(&rd, &tensor, &proposed);
  let gates = evaluate_stitcher_gates(&rd, &candidates, &mirror_reports, &delta);
  let accepted = collect_accepted_updates(&rd, &candidates, &gates);

  if accepted.is_empty() && rd.policies.reject_empty_update_set {
    return StitcherOutcome::Hold(make_hold_report(rd, tensor, fabric_h, gates));
  }

  let tensor_after = apply_coupling_updates(&rd, tensor, &accepted);
  let trace = write_field_tensor_trace(&rd, &tensor_after, &accepted, &gates);
  StitcherOutcome::Completed(write_stitcher_report(
    rd, fabric_h, tensor_after, accepted, gates, trace
  ))
}
```

12 CLI

```
phase-matrix stitch-fabric <cell-cycle.json> --rd stitch-rd.json --out fabric-h.json
phase-matrix stitch-candidates <fabric-h.json> --tensor tensor.json --rd stitch-rd.json --
  out candidates.json
phase-matrix stitch-gate <candidates.json> --tensor tensor.json --rd stitch-rd.json --out
  gates.json
```

```
phase-matrix stitch-apply <candidates.json> --gates gates.json --tensor tensor.json --rd
  stitch-rd.json --out stitch.json
phase-matrix stitch-cycle <cell-cycle.json> --tensor tensor.json --rd stitch-rd.json --
  out stitch.json
phase-matrix stitch-replay <stitch.json>
phase-matrix tensor-inspect <tensor.json>
```

13 Tests

13.1 Unit Tests

1. resonance_fabric_is_content_addressed
2. stitch_candidate_derivation_is_deterministic
3. mirror_consistency_required_for_update
4. tensor_delta_blocks_large_update
5. sticher_gate_fails_closed_without_trace
6. sticher_gate_fails_closed_without_boundary
7. rejected_candidate_keeps_tensor_unchanged
8. accepted_update_changes_tensor_revision
9. coupling_updates_sorted_before_hashing
10. field_tensor_trace_preserves_evidence

13.2 Integration Tests

1. stitch_cycle_happy_path_updates_tensor
2. stitch_cycle_no_candidates_holds
3. stitch_cycle_boundary_violation_keeps_tensor_unchanged
4. stitch_replay_byte_identical
5. cli_stitch_cycle_writes_report
6. cli_stitch_replay_ok

13.3 Negative Tests

1. Ein Fabric-H-Link darf den Tensor niemals direkt mutieren.
2. Ein CouplingUpdate ohne GateReport ist ungueltig.
3. Ein SticherReport mit modifiziertem Tensor-Hash wird erkannt.
4. Ein zu grosser TensorDelta erzeugt Hold oder Reject.
5. Fehlende Evidence verhindert persistente Kopplung, wenn Policy dies verlangt.

14 Definition of Done

Eine Implementierung ist abgeschlossen, wenn:

1. alle Module unter `crates/phase-matrix/src/cell/` existieren oder aequivalent eingebunden sind;
2. alle public Typen dokumentiert sind;
3. alle neuen Reports content-addressed sind;

4. alle Gate-Pfade fail-closed sind;
5. abgelehnte Kandidaten den Tensor byte-identisch unverändert lassen;
6. akzeptierte CouplingUpdates die Tensorrevision deterministisch erhöhen;
7. Replay byte-identische StatcherReports erzeugt;
8. CLI `stitch-cycle`, `stitch-replay` und `tensor-inspect` funktionieren;
9. Tests fuer Happy Path, Hold, Reject, BoundaryViolation und Replay existieren;
10. `cargo test -workspace`, `cargo fmt -all - -check`, `clippy -D warnings` und `cargo doc -no-deps` erfolgreich laufen.

15 Coding-Agent-Auftrag

PATCH-ID: PHASEMATRIX-HIVEMIND-03.1

TITLE: Dual-Fabric Field-Tensor Stitch Layer

GOAL:

Implementiere einen additiven Statcher-Patch fuer phase-matrix. Fabric-H repräsentiert ephemere Resonanzlagen aus vorhandenen ResonanceClusters und ClusterTraces. Fabric-T ist ein persistenter FieldTensorState mit CouplingMatrix. Der Statcher darf Fabric-T nur ueber gegatete, tracebare CouplingUpdates veraendern.

MUST:

- Neue Module unter `crates/phase-matrix/src/cell/` anlegen:
`field_tensor.rs`, `resonance_fabric.rs`, `coupling_update.rs`, `stitcher_gate.rs`,
`stitcher.rs`, `tensor_delta.rs`, `mirror_consistency.rs`, `field_tensor_trace.rs`.
- Typen aus dieser Spezifikation implementieren.
- `StitchRunDescriptor`, `StitchThresholds` und `StitchPolicies` bereitstellen.
- `StitcherGate` fail-closed implementieren.
- `TensorDeltaReport` und `MirrorConsistencyReport` vor Update-Anwendung berechnen.
- `Rejected candidates` duerfen `FieldTensorState` nicht veraendern.
- `Accepted CouplingUpdates` muessen `Trace/Evidence/GateReport` referenzieren.
- CLI `stitch-cycle`, `stitch-replay` und `tensor-inspect` bereitstellen.
- Tests gemaess Spezifikation liefern.

MUST NOT:

- Fabric-H darf Fabric-T niemals direkt mutieren.
- Keine externen Commits oder finalen Artefakte erzeugen.
- Keine platform-floats im `Audit-/Gate-/Hash-Pfad` verwenden.
- Keine nichtdeterministische Randomness im `Replay-Pfad` verwenden.

SHOULD:

- Bestehende v0.3-Reports als Input nutzen, ohne sie umzubenennen.
- `CouplingUpdates` vor Hashing sortieren.
- README um eine kurze PhaseMatrix-03.1-Statuszeile und Quickstart erweitern.

16 Schlussinvariante

$$FabricH(t) \xrightarrow{StitcherGate} CouplingUpdate[] \rightarrow FabricT(t+1)$$

Nur wenn:

$$G_{stitch} = 1.$$

Andernfalls:

$$FabricT(t + 1) = FabricT(t).$$

Endinvariante

Die PhaseMatrix darf ephemere Resonanzereignisse beobachten, bewerten und verwerfen. Sie darf persistente Struktur nur dann veraendern, wenn die Veraenderung als CouplingUpdate gegatet, begrenzt, belegt, getraced und replayfaehig ist.
